

Lesson 9 – Questions

[Basic Questions 10]: Fundamental Concepts

1. What does the vocab_size parameter represent, and where is its value anchored?

↳ vocab_size – tokenizer vocabulary tokenlar soni. Ex) vocab_size = 8000 bulsa, model 8000 ta token ID bilan ishlaydi: 0 dan 7999 gacha. Bu qiymat tokenizer vocabulary bilan mos bulishi kerak.

Follow-up: Runtime da OOV string kiritilsa, network crash qiladimi?

Yuq. BPE tokenizer unknown suzni kichikroq subword tokenlarga ajratadi. Shuning uchun modelga baribir vocabulary ichidagi mavjud token ID lar kiradi.

2. Define block_size (context_size) and explain its practical structural limits.

↳ block_size - model bir vaqtni uzida kura oladigan maximal context uzunligi. Ex) block_size = 256 bulsa, model bir sequence ichida maksimal 256 ta tokenni context sifatida ishlata oladi. 256 tadan uzun matnni bitta sequence sifatida tuliq ishlata olmaydi.

Follow-up: Nega projectda 256 qilib quyilgan, 4096 emas?

Chunki bu experiment loyihasi. 256 GPU memoryni kamroq ishlatadi, trainingni tezlashtiradi va architecture ni urganish uchun yetarli. 4096 esa ancha kup compute budjet va memory talab qiladi.

3. What is n_embd and how does it translate input token streams?

↳ n_embd - har bir token model ichida nechta ulchamli vector bilan ifodalanishini bildiradi. Ex) n_embd = 384 bulsa, har bir token 384-dimensional vektorga aylanadi. Model keyingi hisob kitoblarni shu vectorlar orqali bajaradi.

Follow-up: n_embd ni 384 dan 128 ga tushirsak nima buladi?

Model kichrayadi, tezroq ishlaydi va kamroq memory ishlatadi. Lekin representation space kichraygani uchun model murakkab semantic relationshiplarni ifodalash uchun kamroq hajmga ega buladi.

4. What happens architecturally as the model depth (n_{layer}) scales up?

- n_{layer} - Transformer blocklarining soni. Layer kupaygani sari model malumotni kuproq bosqichda qayta ishlaydi va murakkabroq patternlarni urganishi mumkin. lekin training ham sekinlashadi va kuproq memory kerak buladi.

Follow-up: 1-2 layer va 5-6 layer urtasidagi farq nima?

1-2 layer odatda oddiyroq va local patternlarni qayta ishlaydi. 5-6 layer esa oldingi layerlarda hosil bulgan representationlarni yanada chuqurroq qayta ishlab, murakkabroq context va relationshiplarni urganishi mumkin. har bir layer faqat bitta vazifani bajarmaydi.

5. What are Attention Heads (n_{head}) and why are they run in parallel?

- Attention head - attention mechanismni alohida qismi. Har bir head inputdagi turli relationshiplarga etibor berishni urganishi mumkin. Ular parallel ishlaydi, keyin natijalari birlashtiriladi. Shu sababli model bir vaqtning uzida turli relationshiplarni kurishi mumkin.

Follow-up: Nega n_{embd} n_{head} ga qoldiqsiz bo'linishi kerak?

Chunki embedding dimension headlar orasida teng taqsimlanadi.

Ex) $n_{\text{embd}} = 384$ va $n_{\text{head}} = 6$ bulsa, har bir head $384 / 6 = 64$ dimension oladi.

Agar qoldiqsiz bulinmasa, headlarga teng integer dimension berib bulmaydi va PyTorch tensorlarni kerakli shape ga ajratishda shape/runtime error beradi.

6. What is the regularizing effect of dropout: 0.1 during the pretraining cycle?

- dropout = 0.1 training vaqtida activationlarning taxminan 10 foizini vaqtincha uchiradi. Bu modelning training datasetni haddan tashqari yodlab olishining oldini olishga va overfittingni kamaytirishga yordam beradi.

Follow-up: Live inference da dropout ishlashi kerakmi?

Yuq. Inference vaqtida dropout uchiriladi, chunki model urgangan barcha informationdan foydalanishi kerak. PyTorch da `model.eval()` ishlatilganda dropout deactivation qilinadi.

7. What is the primary function of the Feed-Forward Network (FFN) layer?

➡ FFN - Transformer block ichidagi token representationni qayta ishlaydigan neural network qismi. U representationni avval kengaytiradi, Ex) 384 → 1536, keyin yana 384 ga qaytaradi. Urgangan bilimlarni saqlab qoladi

Follow-up: Nega hidden dimension 4x kengaytiriladi?

4x kengaytirish FFN ichida kuproq representation space va computation beradi. Bu modelga token informationni murakkabroq transformatsiya qilish imkonini beradi. Keyin dimension yana asosiy n_{embd} qiymatiga qaytariladi

8. Explain the meaning and structural layout of tie_weights: true

➡ tie_weights = true bulsa, token Embedding va LM Head bir xil weight matrix dan foydalanadi. Oddiy holatda ular ikkita alohida katta matrix buladi. Weight tying esa bitta matrixni ikkala joyda ishlatadi va parameter sonini kamaytiradi.

Follow-up: 10M parameterli architecture da qancha parameter tejaladi?

Agar vocab_size = 8000 va n_{embd} = 384 bulsa:

$$8000 \times 384 = 3,072,000$$

Yani, taxminan 3.07 million parameter tejaladi. Bu alohida LM Head matrixini saqlamaslik hisobidan buladi.

9. How do you evaluate step zero performance via the Random-init loss check?

➡ Random-init loss check training boshlanishidan oldin modelning random holatdagi lossini tekshiradi. Model hali hech narsa urganmagan bulsa, 8000 ta tokenning har birini taxminan teng ehtimol bilan tanlaydi.

Follow-up: Nega baseline loss taxminan 8.98 yoki 8.99 bulishi kerak?

Random modelning tugri tokenni topish ehtimoli taxminan $1/8000$ buladi. Cross-entropy loss: $\ln(1/8000) = \ln(8000) \approx 8.99$

Shuning uchun vocab_size = 8000 bulgan random model uchun boshlangich loss taxminan 8.99 bulishi kutiladi. 8.98 atrofidagi qiymat ham normal kichik farq hisoblanadi.

10. What is 'Greedy Decoding' inside the core generate() pipeline?

➡ Greedy Decoding - model har bir generation stepida probabilitysi eng yuqori bulgan tokenni tanlaydigan usul. Ex) model "cat" uchun 0.60, "dog" uchun 0.25, "bird" uchun 0.15 probability bersa, greedy decoding "cat" ni tanlaydi. Keyingi stepda ham yana eng yuqori probabilityli token tanlanadi.

Follow-up: Greedy generationning asosiy kamchiligi nima va qachon production da ishlatiladi?

Asosiy kamchiligi - model faqat hozirgi stepdagi eng yuqori probabilityni tanlaydi va boshqa ehtimolli variantlarni hisobga olmaydi. Shu sababli output bazida predictable, takroriy yoki kamroq creative bulishi mumkin. Production da esa tez, deterministic va predictable output kerak bulgan holatlarda ishlatiladi.

[Intermediate Questions 10]: Architecture & Mathematical Logic

11. Causal Self-Attention va classic Global Attention farqi nima?

➡ Causal Self-Attention token faqat uzidan oldingi va uzining tokenlarini kura oladi. Ex, "I am learning AI" jumlasida "learning" tokeni "I", "am" va "learning"ni kura oladi, lekin hali kelmagan "AI" tokenini kura olmaydi.

Global Attention esa barcha tokenlarni bir-biri bilan kura oladi. Yani "learning" tokeni "AI" tokenini ham kura oladi.

Follow-up: Agar causal mask bulmasa nima buladi?

Model kelajakdagi tokenlarni ham kura boshlaydi. Masalan, "I am learning" uchun "learning"ni bashorat qilayotganda target javobni oldindan kurib oladi. Shuning uchun loss juda yaxshi, hatto juda past kurinishi mumkin, lekin bu haqiqiy language modeling emas. Model kelajakni yoddan emas, bevosita kurib bashorat qiladi.

12. Nega parameter soni token budget bilan mos bulishi kerak?

➡ Model juda katta bulsa, uni yetarli token bilan train qilish qiyin va model parametrlaridan tulliqlik foydalanilmaydi. Model juda kichik bulsa esa katta datasetdagi informationni yetarlicha sigdira olmaydi. Shuning uchun model size va training tokenlar soni bir-biriga mos tanlanadi.

Follow-up: 100M token uchun Chinchilla qoidasi buyicha optimal parameter qancha?

Chinchilla qoidasi taxminan 20 token / 1 parameter deydi.

$100M / 20 = 5M$ parameters.

Demak, taxminan 5 million parameter optimal scale buladi.

13. Nega weightlar 0 emas, odatda 0.02 standard deviation bilan initialize qilinadi?

→ Weightlarni random qiymatlar bilan boshlash kerak, chunki turli neuronlar turlicha signal olishi kerak. 0 bilan boshlansa, barcha neurallar bir xil ishlaydi va bir-biridan farq qila olmaydi.

Follow-up: Agar barcha Linear weightlar 0 qilinsa, backpropagation nima buladi?

Gradientlar bir xil bulib qoladi. Barcha neuronlar bir xil qiymatlarni oladi va bir xil gradient oladi. Natijada ular bir-biridan ajralib, turli patternlarni urganmaydi. Bu symmetry muammosi deyiladi.

14. Pre-Normning afzalligi nima?

→ Pre-Normda LayerNorm asosiy sub-layerdan oldin turadi:

$x \rightarrow \text{LayerNorm} \rightarrow \text{Attention/FFN} \rightarrow \text{Residual}$

Bu deep networkda gradientning barqarorroq utishiga yordam beradi va trainingni osonlashtiradi.

Follow-up: Post-Norm 32+ layerda qanday muammo yaratishi mumkin?

Post-Normda normalization residualdan keyin keladi. Layerlar chuqurlashgani sari gradientlar beqaror bulishi yoki vanishing/exploding muammolari kuchayishi mumkin. Shu sababli juda deep Transformerlarni train qilish qiyinlashadi.

15. Residual Connection gradient uchun nima qiladi?

→ Residual connection gradient uchun shortcut yaratadi:

$x \rightarrow \text{Sub-layer} \rightarrow + \rightarrow \text{output}$

Backward paytida gradient tugridan-tugri x tomonga ham utishi mumkin. Shu sababli gradientning deep network orqali yuqoriga qaytishi osonlashadi.

Follow-up: 6-layer stackda residual olib tashlansa gradient qanday yuradi?

Gradient endi shortcutga ega bulmaydi. U:

Layer 6 $\rightarrow$ Layer 5 $\rightarrow$ Layer 4 $\rightarrow$ Layer 3 $\rightarrow$ Layer 2 $\rightarrow$ Layer 1

orqali ketma-ket utishi kerak. Har bir layerdagi transformation gradientni kichraytirishi yoki kattalashtirishi mumkin. Shuning uchun deep stackda gradient vanishing yoki exploding xavfi ortadi.

16. Causality Test nima tekshiradi?

→ Causality Test model haqiqatan ham kelajakdagi tokenni kurmayotganini tekshiradi. Biror future tokenni uzgartirib, oldingi tokenning outputi uzgarmasligi kerak.

Follow-up: Agar t indexdagi future tokenni uzgartirish $t-1$ indexdagi logitsni uzgartirsa, nima buzilgan?

Causal masking buzilgan. Muammo Attention sub-layerdagi causal maskda buladi, chunki $t-1$ tokeni future t tokenini kurmasligi kerak.

17. Nega FFN har bir position uchun mustaqil ishlashi muhim?

→ Chunki tokenlar orasidagi interactionni Attention bajaradi. FFN esa har bir tokenning representationini alohida qayta ishlaydi.

Follow-up: Attention va FFN tokenlar bilan qanday ishlaydi?

Self-Attention: tokenlar bir-biri bilan interaction qiladi.

Ex: $A \leftrightarrow B \leftrightarrow C$

FFN: har bir token alohida qayta ishlanadi.

Masalan:

$A \rightarrow \text{FFN} \rightarrow A \text{ yangi representation}$

$B \rightarrow \text{FFN} \rightarrow B \text{ yangi representation}$

$C \rightarrow \text{FFN} \rightarrow C \text{ yangi representation}$

Shuning uchun Attention informationni tokenlar orasida aralashtiradi, FFN esa har bir token representationini chuqurroq transformatsiya qiladi.

18. Overfit-one-batch verification nima uchun qilinadi?

→ Maqsad model architecture va training loop ishlayotganini tekshirish. Bitta kichik batchni modelga qayta-qayta beramiz. Model uni deyarli yodlab, lossni juda past qiymatga tushirishi kerak.

19. Nega generation loop ichida `idx_cond = idx[: -block_size:]` qilinadi?

↳ Chunki model faqat `block_size` miqdoridagi contextni kura oladi. Sequence uzayib ketganda, eng oxirgi tokenlarni qoldirib, eski tokenlarni tashlab yuborish kerak.

Follow-up: Prompt 300 token bulsa, `block_size = 256` bulsa nima buladi?

Model faqat oxirgi 256 tokenni kura oladi.

Yani: $300 - 256 = 44$ token

model uchun completely invisible buladi.

20. Nega configlar YAML faylda saqlanadi, script ichida hardcode qilinmaydi?

↳ Chunki configuration va code bir-biridan ajratiladi.

Ex: configs/run_01.yaml ichida model size, learning rate, batch size, layer soni va boshqa parametrlar turadi.

Shunda kodni uzgartirmasdan boshqa experiment qilish mumkin.

Follow-up: Bu distributed engineering teamda reproducibilityni qanday taminlaydi?

Hamma bir xil YAML configurationdan foydalansa, bir xil parametrlar bilan experimentni qayta ishga tushira oladi.

Ex) teamdagi hamma: run_01.yaml dan foydalansa, model architecture, learning rate, batch size va boshqa settinglar bir xil buladi.

Shuning uchun kim qanday configuration bilan experiment qilganini aniq saqlash va keyinchalik aynan usha runni takrorlash mumkin.

[Advanced Questions 10]: Industry Standards & Systems Thinking

21. Nega KV Cache autoregressive LLM servingda juda muhim?

KV Cache oldingi tokenlar uchun hisoblangan Key va Value qiymatlarini saqlab turadi. Shuning uchun har yangi token kelganda oldingi tokenlarning K va V qiymatlarini boshidan hisoblash shart bulmaydi. Bu generationni ancha tezlashtiradi.

Follow-up: KV Cache ishlatilmasa nima buladi?

Har yangi token uchun oldingi barcha tokenlarning Key va Value qiymatlari qayta hisoblanadi. Sequence uzaygani sari computation juda kupayadi va generation latency oshadi.

22. Nega GQA MHA ga qaraganda katta batch servingda samaraliroq bulishi mumkin?

MHA da har bir Query head uchun alohida Key va Value head mavjud. GQA da bir nechta Query head bir xil K va V headlardan foydalanadi. Natijada KV Cache kichrayadi va bir vaqtda kuproq requestni VRAMga sigdirish mumkin.

Follow-up: Masalan, 32 Query head va 8 KV head bulsa nima anglatadi?

32 ta Query head mavjud, lekin faqat 8 ta K va V head mavjud. Demak, har 4 ta Query head bitta KV head guruhidan foydalanadi. Bu KV Cache hajmini MHA ga nisbatan sezilarli kamaytiradi.

23. Nega Flash Attention oddiy Attentiondan samaraliroq?

Oddiy Attentionda katta Attention matrix hosil qilinadi va GPU memoryga kup intermediate data yoziladi. Flash Attention esa hisoblashni kichik blocklarga bulib, GPU memorydan samaraliroq foydalanadi. Natijada memory traffic kamayadi va Attention tezroq ishlashi mumkin.

Follow-up: Flash Attentionning asosiy foydasi model matematikasini uzgartirishmi?

Yuq. Asosiy Attention formulasi saqlanadi. Asosiy farq hisoblashni GPU memory va computation uchun samaraliroq tashkil qilishda.

24. Nega mixed precision training ishlatiladi?

Mixed precisionda ayrim hisob-kitoblar BF16 yoki FP16 kabi kichik precisionda, ayrim muhim state esa yuqoriroq precisionda bajariladi. Bu GPU memoryni kamaytiradi va trainingni tezlashtiradi.

Follow-up: BF16 nega LLM training uchun qulay?

BF16 katta dynamic rangega ega va FP32 ga qaraganda kamroq memory ishlatadi. Shu sababli katta neural networklarni training qilishda precision va efficiency urtasida yaxshi balans beradi.

25. Nega gradient accumulation katta effective batch size olish uchun ishlatiladi?

GPU VRAMga katta batch sigmasa, kichik batchlar ketma-ket hisoblanadi va ularning gradientlari yigilib boriladi. Bir nechta micro-batchdan keyin optimizer update qilinadi. Shu orqali fizik batch kichik bulsa ham katta effective batch hosil qilinadi.

Follow-up: Masalan, batch size 8 va accumulation 4 bulsa effective batch qancha?

$8 \times 4 = 32$. Demak, optimizer har 4 ta micro-batchdan keyin update qilsa, effective batch size 32 ga teng buladi.

26. Nega learning rate warmup katta LLM trainingda ishlatiladi?

Trainingning dastlabki bosqichida weightlar hali random holatda buladi. Learning rate darhol katta qiymatda bulsa, gradient update juda keskin bulishi mumkin. Warmup learning rate ni asta-sekin oshirib, trainingni barqarorroq boshlashga yordam beradi.

Follow-up: Warmup tugagandan keyin nima buladi?

Learning rate belgilangan maksimal qiymatga yetadi va keyin schedulerga qarab kamayishi mumkin. Masalan, cosine decay ishlatilsa, training davomida learning rate asta-sekin pasayadi.

27. Nega validation loss training lossdan farq qiladi?

Training loss model training datasetdagi tokenlarni qanchalik yaxshi bashorat qilayotganini bildiradi. Validation loss esa model ilgari update qilinmagan boshqa datasetda qanday ishlayotganini kursatadi.

Follow-up: Training loss tushib, validation loss kutarilsa nima buladi?

Bu overfitting belgisi bulishi mumkin. Model training data patternlarini juda yaxshi urganadi, lekin yangi validation data uchun generalization yomonlashadi.

28. Nega perplexity language model uchun muhim metric?

Perplexity modelning keyingi tokenni qanchalik yaxshi bashorat qilayotganini loss asosida ifodalaydi. Perplexity odatda $PPL = \exp(\text{loss})$ formula orqali olinadi. Loss qancha past bulsa, perplexity ham shuncha past buladi.

Follow-up: Loss = 1 bulsa perplexity qancha?

$\exp(1) \approx 2.72$. Bu modelning prediction uncertainty darajasini intuitivroq ifodalashga yordam beradi.

29. **Nega model parameter soni uning haqiqiy sifatini yolgiz belgilamaydi?**

Parameter soni model capacity haqida malumot beradi, lekin natija dataset sifati, token budget, architecture, optimization va training durationga ham bogliq. Shuning uchun 100M parameterli model har doim 50M parameterli modeldan yaxshi buladi deb aytib bulmaydi.

Follow-up: Kichik model katta modeldan yaxshi natija berishi mumkinmi?

Ha, agar kichik model sifatli datasetda tugri training qilingan bulsa va katta model kam yoki sifatsiz data bilan training qilingan bulsa, kichik model ayrim vazifalarda yaxshiroq natija berishi mumkin.

30. **Nega LLM production servingda quantization ishlatiladi?**

Quantization model weightlarini FP32 yoki BF16 kabi yuqori precisiondan INT8 yoki INT4 kabi kichikroq representationga utkazadi. Bu model hajmini va VRAM talabini kamaytiradi. Natijada model consumer GPU yoki edge hardwareda ishlashi osonlashadi.

Follow-up: INT8 yoki INT4 qilish model sifatini buzmaydimi?

Biroz quality loss bulishi mumkin, chunki original numerical precision kamayadi. Lekin yaxshi quantization usullari bu yuqotishni minimal darajada saqlashga harakat qiladi. Asosiy trade-off kamroq memory va tezroq inference evaziga ozgina precision yuqotishidir.